

گزارش نهایی پروژه فشرده‌سازی داده‌ها

پیاده‌سازی و ارزیابی Huffman، LZW، Adaptive Huffman و RLE با Python

معرفی و هدف پروژه

این پروژه با هدف پیاده‌سازی چند روش فشرده‌سازی بدون اتلاف و بررسی ارتباط آن‌ها با مفاهیم نظریه اطلاعات انجام شد. شرط اصلی آن بود که داده پس از رمزگشایی دقیقاً و به‌صورت بایت‌به‌بایت با ورودی یکسان باشد. هسته پروژه الگوریتم هافمن است و در کنار آن LZW، هافمن تطبیقی و RLE نیز پیاده‌سازی و مقایسه شدند.

پیاده‌سازی الگوریتم هافمن

ابتدا فراوانی تمام بایت‌های ورودی محاسبه و هر نماد به یک گره برگ تبدیل شد. گره‌ها در صف اولویت قرار گرفتند و در هر مرحله دو گره با کمترین فراوانی ادغام شدند. با تکرار این کار، درخت دودویی هافمن تشکیل شد و با اختصاص صفر به یال چپ و یک به یال راست، کد هر نماد به دست آمد.

برای ایجاد فایل فشرده مستقل، علاوه بر جریان بیت، اندازه داده اصلی، جدول فراوانی و تعداد بیت‌های معتبر در Header ذخیره شد. رمزگشا با استفاده از همین اطلاعات، درخت را بازسازی و فایل اصلی را تولید می‌کند. روند ساخت درخت نیز مرحله‌به‌مرحله رسم و به‌صورت تصاویر PNG و یک GIF متحرک ذخیره شد.

الگوریتم‌های تکمیلی

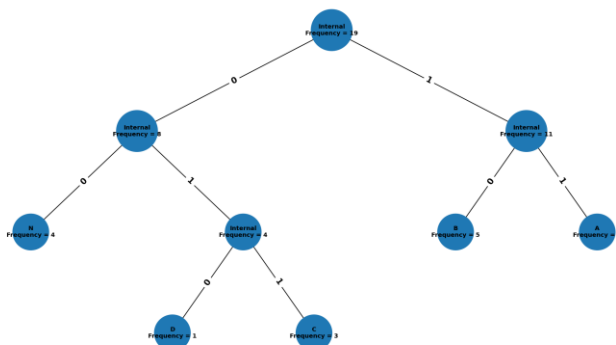
LZW با فرهنگ اولیه ۲۵۶ بیتی و کدهای ثابت ۱۲ بیتی اجرا شد. هافمن تطبیقی بدون اسکن اولیه فراوانی‌ها، مدل خود را پس از هر نماد به‌روزرسانی کرد. RLE برای تصویر BMP سیاه‌وسفید به کار رفت و مازول فشرده‌سازی و بازسازی تصویر با هافمن نیز آماده شد.

ساختار نرم‌افزاری

کدها در ماژول‌های جداگانه برای Huffman، LZW، Adaptive Huffman و RLE سازمان‌دهی شدند. پوشه‌های data، results و tests نیز به‌ترتیب برای ورودی‌ها، خروجی‌های تحلیلی و آزمون‌های صحت در نظر گرفته شدند. یک ابزار خط فرمان نیز برای فشرده‌سازی و بازیابی فایل‌ها تهیه شد.

برای داده انگلیسی، آنتروپی شانون برابر 4.381 و میانگین طول کد هافمن برابر 4.401 بیت بر بایت شد. برای داده فارسی نیز این مقادیر به ترتیب 4.100 و 4.125 بودند. در هر دو حالت رابطه $H \leq L < H + 1$ برقرار و مجموع Kraft برابر یک بود؛ بنابراین کدهای تولیدی پیشوندی و کامل بودند.

بهینگی هافمن: اثبات با استدلال تعویض، خواهر بودن دو نماد کم‌احتمال، کاهش مسئله از n نماد به $n-1$ نماد و استقرای ریاضی انجام شد. نتیجه نشان می‌دهد هافمن در میان کدهای دودویی پیشوندی، کمترین میانگین طول کد را تولید می‌کند.



درخت نهایی هافمن برای رشته BANANA_BANDANA

آزمون و اعتبارسنجی

پروژه با ورودی خالی، ورودی تک‌نمادی، داده‌های تکراری، تمام مقادیر ممکن بایت و داده‌های تصادفی آزمایش شد. افزون بر رفت‌و برگشت صحیح، Prefix-Free بودن کدها، نابرابری Kraft و کران آنتروپی نیز بررسی شدند. هر پنج آزمون خودکار با موفقیت عبور کردند. در فایل‌های فارسی و انگلیسی، خروجی بازبازی شده در روش‌های Huffman، LZW و Adaptive Huffman دقیقاً با فایل اصلی برابر بود. این برابری بر اساس مقایسه مستقیم بایت‌ها انجام شد و به یکسان بودن ظاهری متن محدود نبود.

معیار صحت: `original_bytes == restored_bytes` برای تمام الگوریتم‌ها و تمام داده‌های آزمایشی.

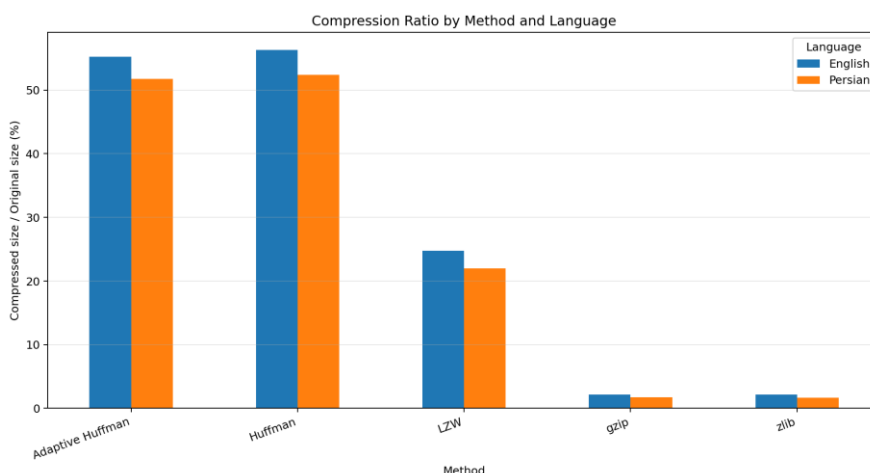
خروجی‌های گرافیکی

برای تحلیل بهتر، نمودار مقایسه نسبت فشرده‌سازی روش‌ها، نمودار آنتروپی و میانگین طول کد، نمودار اندازه فایل اصلی و هافمن، تصاویر مراحل ساخت درخت و خروجی‌های تصویر RLE ایجاد شدند. این نمودارها امکان مقایسه مستقیم رفتار الگوریتم‌ها را فراهم کردند.

متن انگلیسی با اندازه ۳۳،۱۱۹ بایت و متن فارسی با اندازه ۵۱،۱۱۹ بایت. هافمن اندازه آن‌ها را به ترتیب به ۱۸،۶۳۶ و ۲۶،۷۹۳ بایت کاهش داد؛ یعنی نسبت اندازه فشرده به اصلی حدود ۵۶،۳ درصد برای انگلیسی و ۵۲،۴ درصد برای فارسی بود.

LZW به دلیل استفاده از توالی‌های تکراری، نسبت به هافمن نتیجه بهتری داشت و نسبت اندازه را به 24.8 درصد برای انگلیسی و 22.0 درصد برای فارسی رساند. هافمن تطبیقی نتیجه‌ای نزدیک به هافمن ایستا داشت، اما بدون پیش‌خوانی کامل داده عمل کرد.

gzip و zlib بهترین نتیجه را ثبت کردند و نسبت اندازه را در متن‌های آزمایشی تا محدوده 2.2 تا 1.7 درصد کاهش دادند. علت اصلی این برتری، استفاده هم‌زمان از شناسایی تکرارهای طولانی و کدگذاری آماری است.



مقایسه نسبت اندازه فایل فشرده به فایل اصلی؛ مقدار کمتر بهتر است.

آزمایش تصویر

در تصویر سیاه‌وسفید، RLE حجم ۶۵،۵۳۶ بایت داده خام پیکسل را به ۱،۲۵۸ بایت رساند و حدود 98.1 درصد صرفه‌جویی ایجاد کرد. وجود نواحی بزرگ از پیکسل‌های یکسان دلیل اصلی این عملکرد بود. برای تکمیل پروژه، فشرده‌سازی و بازسازی همان تصویر با هافمن نیز به صورت مستقل طراحی شد.

نتیجه نهایی::

پروژه نشان داد که انتخاب روش فشرده‌سازی به ساختار داده بستگی دارد. هافمن برای ساخت کد پیشوندی بهینه بر اساس فراوانی مناسب است، LZW از الگوهای تکراری بهره می‌برد، هافمن تطبیقی برای پردازش تک‌گذره کاربرد دارد و RLE روی تکرارهای متوالی طولانی بسیار مؤثر است. gzip و zlib نیز به دلیل ترکیب چند روش، در متن‌های تکراری بهترین فشرده‌سازی را ارائه کردند.